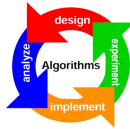


Estrategias algorítmicas

José A. Rodríguez Melquiades

28 de junio de 2026

Programación dinámica



1 Problema de coleccionar monedas

Problema de coleccionar monedas

- **Monedas**¹: Colocar en las celdas de un tablero de dimensión $n \times m$, no más de una moneda por celda.

	1	2	3	4	5	6
1					●	
2		●		●		
3				●		●
4			●			●
5	●				●	

Figura 1: Monedas para coleccionar sobre un tablero.

¹Coin-collecting problem

- **Robot:** Localizado en la celda superior izquierda del tablero, necesita coleccionar tantas monedas como sea posible y llevarlos a la celda inferior derecha.

- **Robot:** Localizado en la celda superior izquierda del tablero, necesita coleccionar tantas monedas como sea posible y llevarlos a la celda inferior derecha.
- **En cada paso:** El robot se mueve una celda a la derecha o una celda abajo desde su localización actual. Cuando el robot visita una celda con una moneda, él siempre la recoge.

- **Robot:** Localizado en la celda superior izquierda del tablero, necesita coleccionar tantas monedas como sea posible y llevarlos a la celda inferior derecha.
- **En cada paso:** El robot se mueve una celda a la derecha o una celda abajo desde su localización actual. Cuando el robot visita una celda con una moneda, él siempre la recoge.
- **Problema:** Desarrollar un algoritmo para hallar el número máximo de monedas que el robot pueda coleccionar y el camino que sigue.

- Formalización del problema:

- Formalización del problema:
 - ▶ $F(i, j)$: Monedas que el robot colecciona para llevarlo a (i, j) del tablero.

- Formalización del problema:

- ▶ $F(i, j)$: Monedas que el robot colecciona para llevarlo a (i, j) del tablero.
 - ★ Un robot puede alcanzar desde la celda adyacente $(i - 1, j)$ que esta arriba o desde la celda adyacente $(i, j - 1)$ que esta a la izquierda de él.

- **Formalización del problema:**

- ▶ **$F(i, j)$:** Monedas que el robot colecciona para llevarlo a (i, j) del tablero.
 - ★ **Un robot puede alcanzar** desde la celda adyacente $(i - 1, j)$ que esta arriba o desde la celda adyacente $(i, j - 1)$ que esta a la izquierda de él.
- ▶ Cantidad de monedas que pueden ser llevadas a esas celdas:

$$F(i - 1, j) \quad F(i, j - 1) \quad (1)$$

- **Formalización del problema:**

- ▶ **$F(i, j)$:** Monedas que el robot colecciona para llevarlo a (i, j) del tablero.
 - ★ **Un robot puede alcanzar** desde la celda adyacente $(i - 1, j)$ que esta arriba o desde la celda adyacente $(i, j - 1)$ que esta a la izquierda de él.
- ▶ Cantidad de monedas que pueden ser llevadas a esas celdas:

$$F(i - 1, j) \qquad F(i, j - 1) \qquad (1)$$

- ▶ **Nota:**

- **Formalización del problema:**

- ▶ **F (i, j):** Monedas que el robot colecciona para llevarlo a (i, j) del tablero.
 - ★ **Un robot puede alcanzar** desde la celda adyacente (i - 1, j) que esta arriba o desde la celda adyacente (i, j - 1) que esta a la izquierda de él.
- ▶ Cantidad de monedas que pueden ser llevadas a esas celdas:

$$F(i - 1, j) \qquad F(i, j - 1) \qquad (1)$$

- ▶ **Nota:**
 - ★ No existen celdas adyacentes encima de las celdas de la primera fila; así como no existen celdas adyacentes a la izquierda de las celdas de la primera columna, es decir, que para esas celdas, asumir que

$$F(i - 1, j) \qquad F(i, j - 1) \qquad (2)$$

son iguales a 0, para sus vecinos no existentes.

- Formalización del problema (cont):

- Formalización del problema (cont):

- ▶ **Por lo tanto:** El mayor número de monedas que el robot puede traer a la celda (i, j) es el máximo (1) mas una posible moneda en la celda (i, j) misma:

$$F(i, j) = \max \{F(i-1, j), F(i, j-1)\} + c_{ij}, \forall i \in [1, n], j \in [1, m] \quad (3)$$

$$F(0, j) = 0, \forall j \in [1, m]; F(i, 0) = 0, \forall i \in [1, n] \quad (4)$$

Donde:

$$c(i, j) = \begin{cases} 1 & \text{Si existe una moneda en la celda (i,j)} \\ 0 & \text{Caso contrario} \end{cases}$$

- **Formalización del problema** (cont):

- ▶ **Por lo tanto:** El mayor número de monedas que el robot puede traer a la celda (i, j) es el máximo (1) mas una posible moneda en la celda (i, j) misma:

$$F(i, j) = \max \{F(i-1, j), F(i, j-1)\} + c_{ij}, \forall i \in [1, n], j \in [1, m] \quad (3)$$

$$F(0, j) = 0, \forall j \in [1, m]; F(i, 0) = 0, \forall i \in [1, n] \quad (4)$$

Donde:

$$c(i, j) = \begin{cases} 1 & \text{Si existe una moneda en la celda (i,j)} \\ 0 & \text{Caso contrario} \end{cases}$$

- **Usando las fórmulas:** Llenar el tablero $n \times m$ de $F(i, j)$ valores fila por fila o columna por columna.

- Algoritmo:

RobotCoinCollection($C[1..n, 1..m]$)

//Aplicar PD para obtener el mayor número monedas que el robot pueda
//coleccionar en un tablero $n \times m$, empezando en $(1, 1)$ y haciendo
//movimientos hacia la derecha y hacia abajo desde la esquina superior
//izquierda hacia la esquina inferior derecha

Input: Matriz $C[1..n, 1..m]$ cuyos elementos son iguales a 1 y 0
para celda con y sin una moneda.

Output: Mayor número de monedas que un robot puede llevar a la celda (n, m)

- (1) $F[1, 1] := C[1, 1]$
- (2) **FOR** $j := 2 : m$
- (3) $F[1, j] := F[1, j - 1] + C[1, j]$
- (4) **FOR** $i := 2 : n$
- (5) $F[i, 1] := F[i - 1, 1] + C[i, 1]$
- (6) **FOR** $j := 2 : m$
- (7) $F[i, j] := \max(F[i - 1, j], F[i, j - 1]) + C[i, j]$
- (8) **Return** $F[n, m]$

● Ejemplo

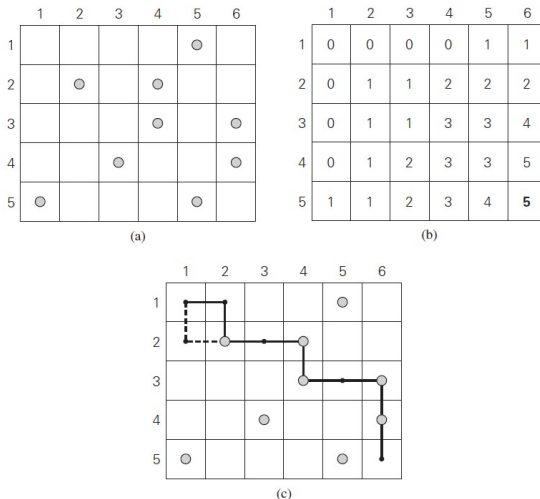


Figura 2: (a) Monedas a coleccionar. (b) Resultados del algoritmo PD. (c) Dos caminos para coleccionar 5 monedas.

- **Ejercicio:**

Dada la sgte. tabla, ¿cuántas monedas podrá recoger el robot?.

	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

Tabla 1: Problemas solucionados con Programación Dinámica

Problema	Área de aplicación	Idea de solución
Camino mínimo	Grafos y redes	Divide el problema en etapas y calcula el costo mínimo acumulado para llegar a cada nodo.
Asignación óptima de recursos	Ingeniería y gestión	Distribuye recursos limitados entre actividades maximizando beneficios o minimizando costos.
Secuencia común más larga (LCS)	Procesamiento de texto y bioinformática	Encuentra la subsecuencia compartida más larga entre dos secuencias.
Distancia de edición	Lenguaje natural y comparación de cadenas	Calcula el número mínimo de operaciones para transformar una cadena en otra.
Planificación de producción	Manufactura y logística	Determina decisiones óptimas de producción a través de múltiples periodos.
Control óptimo dinámico	Automática y sistemas	Calcula la política de control que minimiza costos o maximiza desempeño.
Inventarios multi-periodo	Investigación de operaciones	Optimiza niveles de inventario considerando demanda futura y costos asociados.
Asignación de agua en agricultura	Gestión de recursos naturales	Distribuye agua entre cultivos y periodos para maximizar rendimiento bajo restricciones hídricas.